01 / THE QUESTION

Is the Web App Your AI Built Really Secure?

h5i is an unified Rust framework for building, verifying, and hacking web apps.

BUILD / RUST KERNELS

VERIFY / LEAN 4

HACK / RUNNING APPS

Software security has two sides

A lesson from my PhD: we need both.

RED-TEAMING

Find Bugs ∃

Find a counterexample in the running application.

FORMAL VERIFICATION

Prove Properties ∀

Prove a stated rule for the modeled logic under explicit assumptions.

03 / THE PRODUCT

h5i: Build, Verify, and Hack Web Apps

01 / BUILD

Composable Rust kernels

Axum-based framework with a separate kernel for application decisions and GPU support for suitable batch workloads.

02 / VERIFY

Lean 4 proofs

Extract Rust logic, prove critical properties, and test proof sensitivity with mutations.

03 / HACK

Agent-led red-teaming

Explore any web app, replay HTTP requests, and turn confirmed findings into CI tests.

HOW IT COMES TOGETHER

h5i unifies building, verification, and red-teaming

A shared CLI and local dashboard connect the work, with sandboxed agent workflows for security testing.

04 / BUILD

Build around a pure Rust kernel

h5i-app separates web and database work from the code that makes application decisions.

01 / AXUM

Receive the request

Handle HTTP and load the current state.

→

02 / RUST KERNEL

Decide what happens

Use a command and snapshot to return planned writes and a reply.

→

03 / STORAGE

Apply the result

Commit the writes and send the response.

WHY THIS DESIGN

The isolated kernel can be translated into Lean 4 for proofs and run on GPUs for suitable batch workloads.

Verify application logic with Lean 4

Charon and Aeneas extract Rust kernel logic; Lean checks the properties developers state.

Rust / kernel excerpt

```rust
if p.author == user
    || is_moderator(&s.moderators, user) {
    Ok((one(Write::DelPost(id)),
        Reply::Done))
} else {
    Err(Error::Forbidden)
}
```

Authorization branch in the same board kernel

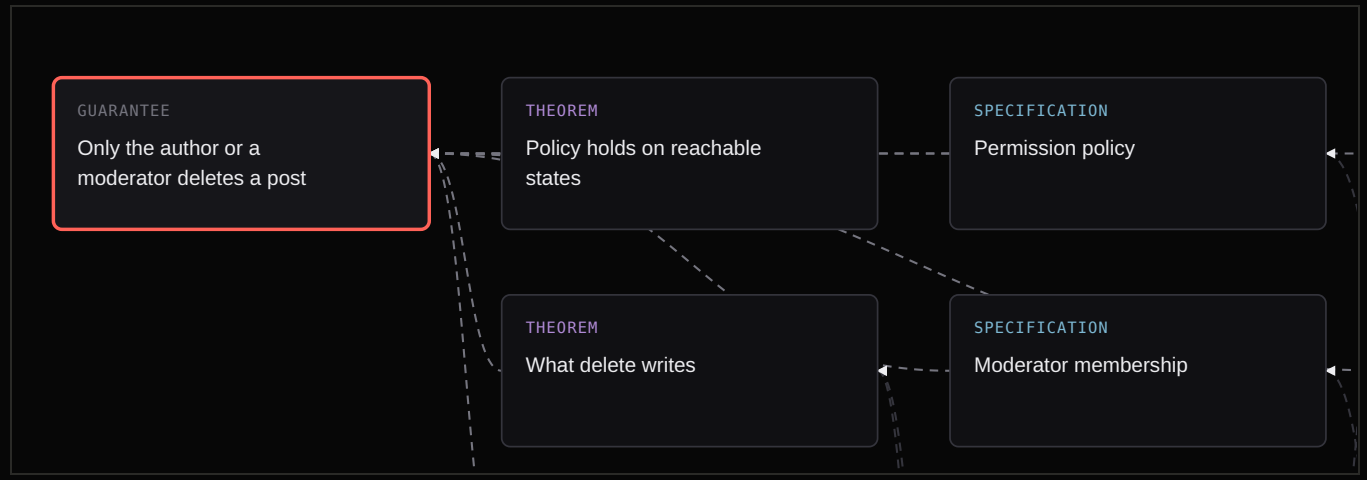
Charon
Aeneas

→

Lean 4 / Theorems.lean · excerpt

```
theorem authorized (a : Principal) (s : Snapshot)
    (c : Command) ws r
    (hinv : Inv (Snapshot.toSt s))
    (h : transition a s c = .ok (.Ok (ws, r))) :
    ∀ w ∈ ws.val, allowed (Snapshot.toSt s) a.user.val w := by -- proof body omitted
```

h5i ui / theorem-spec dependency graph · excerpt



Mutation testing checks whether deliberate kernel bugs break the proofs.

Red-team any web application

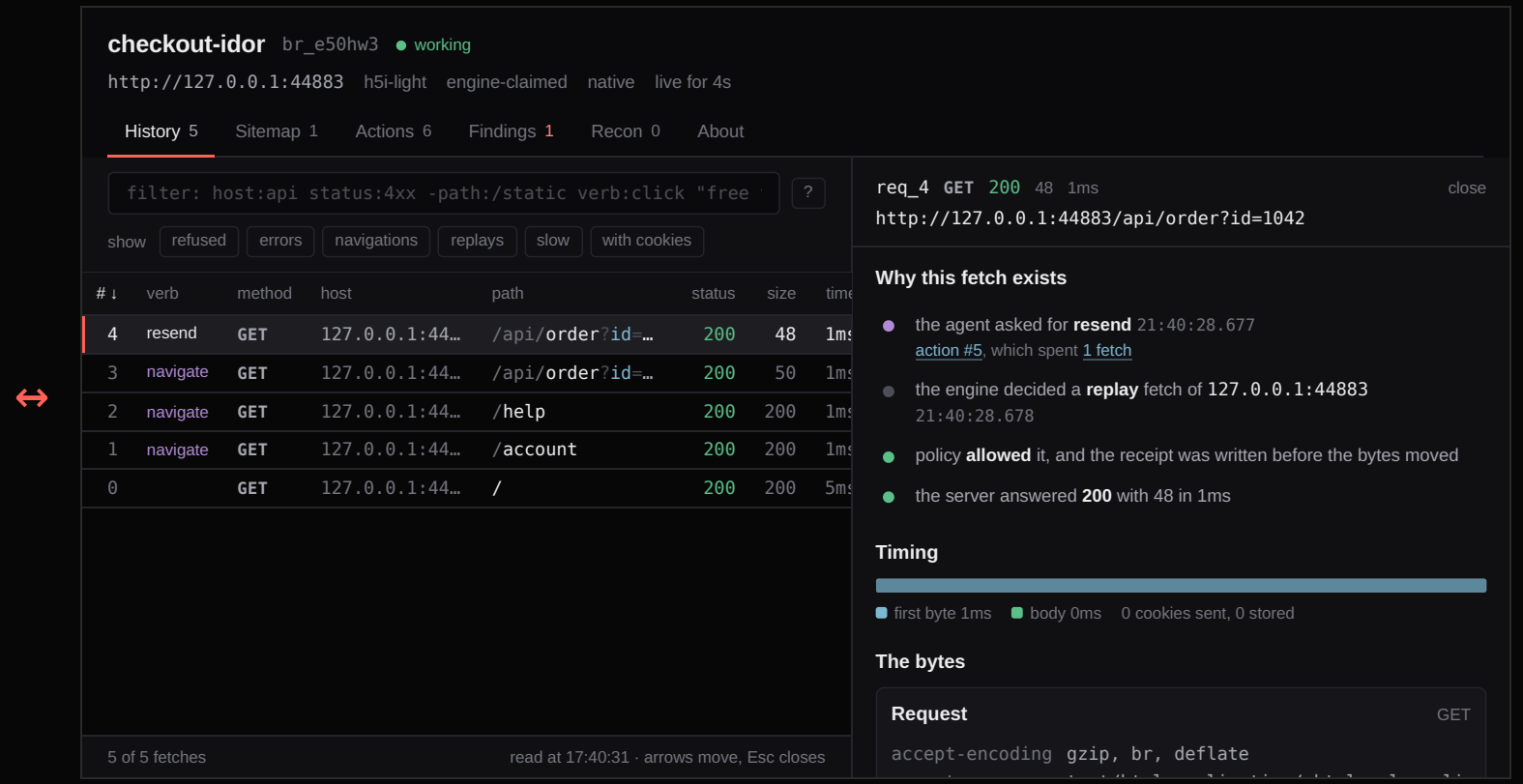
The CLI works with any web stack: the agent explores, replays HTTP requests, and records evidence for review.

AI agent / shell

```
$ h5i browser snapshot
  @e7 Order 1041

$ h5i websec replay req_3 \
    --set query.id=1042
  → req_4 · HTTP 200

$ h5i websec diff res_3 res_4
  owner: Alice → Bob
```



CI REGRESSION `h5i test .h5i-tests/tests/auth.yaml --target https://staging.example.com`

Illustrative shell + actual h5i ui / local demo; confirmed findings can become CI tests

Open-source adoption and results

780+

GITHUB STARS

4.2k+

GITHUB RELEASE ASSET DOWNLOADS

BUILT WITH H5I-APP

o5s · h5i-db

REPORTED SECURITY FINDINGS

rustical advisory · Exograph issue

GitHub API snapshot · October 18, 2026 Repository ↗ Releases ↗

Public interest and reported examples / not customer or revenue metrics

Find bugs. Prove correctness. Build trust.

NEXT

- Work with design partners on real applications
- Improve the workflow
- Build toward the first paid pilots

Start a conversation ↗

Explore h5i ↗